

TP 5

Cuda - First Step

Master 2 SD

2025-2026

HPC

Benoist GASTON

benoist.gaston@univ-rouen.fr

premiers pas avec CUDA

Préliminaire

Pour ces TPs nous utiliserons des notebooks.

Ils peuvent être exécutés sur Google colab research environment qui fournit un accès à des gpus :

<https://colab.research.google.com>

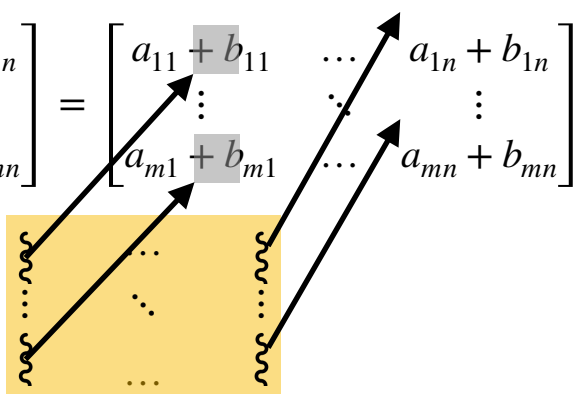
Nécessite un compte Google.

Exercice 1

Jouer les exemples du cours

- `cuda_firstExample.ipynb`
- `deviceQuery.ipynb`
- `sumline.ipynb`

Addition de matrices

$$\begin{bmatrix} a_{11} & \dots & a_{1n} \\ \vdots & \ddots & \vdots \\ a_{m1} & \dots & a_{mn} \end{bmatrix} + \begin{bmatrix} b_{11} & \dots & b_{1n} \\ \vdots & \ddots & \vdots \\ b_{m1} & \dots & b_{mn} \end{bmatrix} = \begin{bmatrix} a_{11} + b_{11} & \dots & a_{1n} + b_{1n} \\ \vdots & \ddots & \vdots \\ a_{m1} + b_{m1} & \dots & a_{mn} + b_{mn} \end{bmatrix}$$


Ex02 Matrix Addition :

`M2SD-HPC-TP05-Ex02-addmat.ipynb`

Le but est d'additionner deux matrices (terme à terme) sur GPU. On ne considérera que des threads organisés en bloc 2D.

1. Compléter le kernel `k_add_mat` afin que chaque thread ne prenne en charge qu'une et une seule addition.
2. Compléter les transferts mémoires entre le host et le device.
3. Appeler le kernel sur les matrices `g_A` et `g_B` en n'utilisant qu'un seul bloc.

On souhaite traiter des matrices plus grosses.

4. Appeler le kernel sur une grille suffisamment grande pour prendre en charge `g_HA` et `g_HB`
5. Sur la base du kernel `k_add_mat`, implémenter le kernel `k_add_huge_mat` dans lequel chaque thread prend en charge `N` additions.
6. Appeler le kernel `k_add_huge_mat` sur les matrices `g_HA` et `g_HB` en utilisant un bloc correctement dimensionné.

premiers pas avec CUDA

Ex03 Matrice Multiplication

M2SD-HPC-TP05-Ex03-multmat.ipynb

Le but est de le produit matriciel de 2 matrices sur GPU. On utilisera des threads organisés en bloc 1D ou 2D et en grille 1D ou 2D.

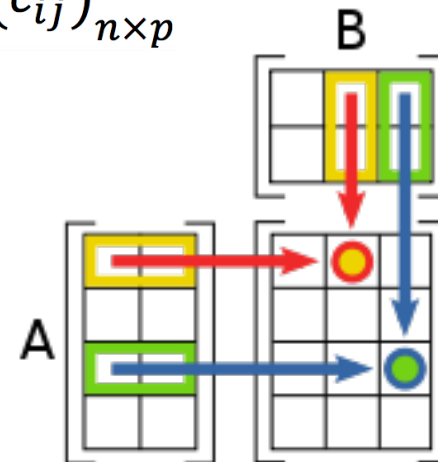
1. Compléter le kernel `k_mult_mat` afin que chaque thread réalise une boucle interne.
2. Appeler le kernel sur les matrices `g_a` and `g_b` en utilisant des blocs et grilles correctement dimensionnés.

```
for i in range(dim1):  
    for j in range(dim2):  
        for k in range(dim3):  
            c[i][j] += a[i][k] * b[k][j]
```

Produit matriciel

$$AB = C = (c_{ij})_{n \times p}$$

$$c_{ij} = \sum_{k=0}^n a_{ik} \times b_{kj}$$



Get started with CUDA

Dot product : M2SD-HPC-TP05-Ex04-dotvec.ipynb

But : réaliser le produit scalaire de deux vecteurs 1D sur GPU. Le produit scalaire est donné par la formule et l'algorithme ci contre.

L'implémentation se fera en deux étapes.

I. Dans un premier temps on ne réalisera sur GPU que les produits terme à terme de chaque vecteur ; la somme est réalisée sur le host.

1. Compléter le kernel `g_dot_vec` afin que chaque thread prenne en charge une et une seule multiplication terme à terme.
2. Appeler le kernel pour les vecteurs `g_va` et `g_vb` en utilisant un seul bloc correctement dimensionné.

II. Trouver un algorithme permettant de réaliser la somme finale sur le device :

- Une solution naïve consiste à réaliser la somme par un seul thread.
- Une seconde solution est d'utiliser une méthode *divide and conquer* en s'inspirant du schéma ci-contre.

NB : on peut opérer si une synchronisation (barrière) des threads d'un bloc par la fonction CUDA : `syncthreads()`

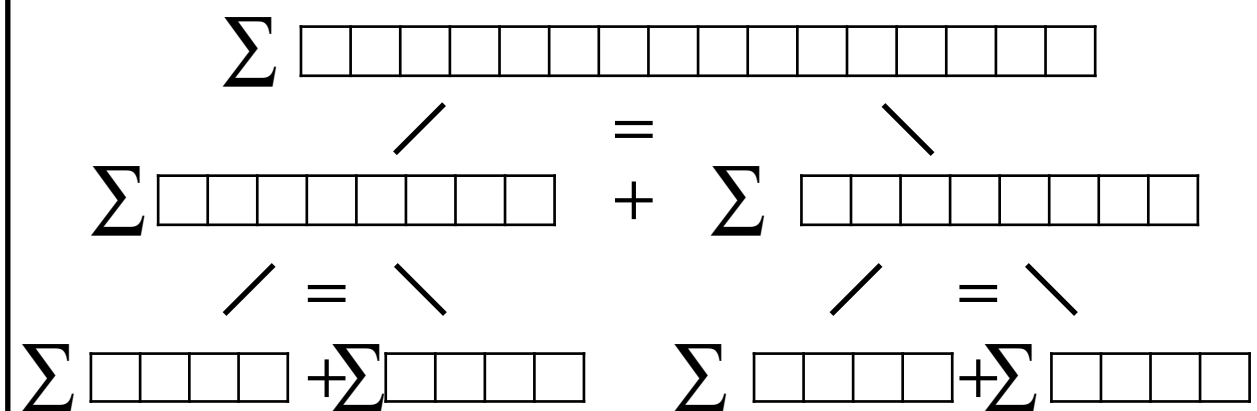
3. Sur la base du kernel `dot_vec` implémenter le kernel `complete_dot_vec` qui réalise les produit terme à terme et la somme finale sur GPU (solution naïve et *divide and conquer*).

4. Quelles sont les limites de ces méthodes de réduction sur GPU ?

$$[a_1, a_2, \dots, a_n] \times [b_1, b_2, \dots, b_n] = a_1b_1 + a_2b_2 + \dots + a_nb_n$$

```
res=np.empty_like(va)
for i in range(va.size):
    res[i]=va[i]*vb[i]
dotprod=sum(res[:])
```

Produit scalaire



Somme parallélisée